ContextRAG

Give LLMs the Context They Need

ContextRAG System Design Playbook

ContextRAG

System Design Playbook

This document complements the ContextRAG Architecture Document.
While the architecture document explains the conceptual design of the ContextRAG platform, this playbook focuses on how the system can actually be built.

The goal is to provide a reference blueprint that can guide implementation decisions, microservice boundaries, scaling strategies, and integration points.

1. Microservice Architecture

A production ContextRAG system can be decomposed into multiple microservices.

Core services may include:

• RAG Orchestrator Service
• Embedding Service
• Knowledge Ingestion Service
• Conversation Summarizer
• LLM Router Service
• Vector Search Service
• API Gateway

Each service can scale independently depending on system workload.

2. Kafka Event Architecture

Kafka acts as the backbone of asynchronous communication.

Typical topics may include:

knowledge-ingestion
conversation-events
conversation-summary

These topics allow ingestion pipelines, summarizers, and indexing services to operate independently without blocking user requests.

3. Vector Database Schema Design

Vector storage in Weaviate should support structured metadata.

Typical schema fields:

- document_id
- project_id
- source_type
- text_chunk
- embedding_vector
- timestamp

This metadata enables more precise retrieval during similarity search.

4. RAG Query Lifecycle

The lifecycle of a single query typically follows these steps:

1. User sends query through UI or IDE plugin
2. RAG service generates embedding
3. Vector search retrieves relevant context
4. Prompt builder merges query with context
5. LLM router sends prompt to selected model
6. Response returned to user

Conversation events are also published to Kafka for summarization.

5. Scaling Strategy

ContextRAG scaling should focus on three main layers:

Compute Layer
• stateless RAG services
• horizontally scalable containers

Data Layer
• distributed vector database clusters

Event Layer
• Kafka partitions enabling parallel ingestion pipelines

This architecture allows ContextRAG to scale from individual developer usage to enterprise-level workloads.

6. IDE Plugin Architecture

IDE plugins can connect directly to the RAG API.

Example flow:

IDE Plugin → API Gateway → RAG Service → Vector DB + LLM

This allows developers to access context-aware AI assistance without leaving their development environment.

7. Developer Workflow

A typical developer workflow might look like this:

1. Developer installs ContextRAG UI or IDE plugin
2. Developer uploads project documentation or repository context
3. ContextRAG ingests knowledge into vector database
4. Developer asks questions about the project
5. ContextRAG retrieves context and augments the prompt
6. LLM generates context-aware response

Over time, conversation summaries enrich the knowledge base.

8. Observability and Monitoring

Operational visibility is essential for large deployments.

Recommended observability layers:

• request tracing for RAG queries
• ingestion pipeline monitoring
• vector database health metrics
• LLM latency monitoring

These signals help operators maintain performance and reliability.

9. Future Extensions

Future versions of ContextRAG may support:

• autonomous engineering agents
• architecture analysis assistants
• observability-driven optimization
• automated documentation generation

These capabilities would extend ContextRAG from a context retrieval platform into a full AI engineering intelligence system.